

Notes on Lec 3

Mohamed Emary

September 20, 2025

To loop over array items in C++:

```
vector<int> arr = {1, 2, 3, 4, 5};  
for (int item : arr) {  
    cout << item << endl;  
}
```

To get the log base 2, use the `log2` function from the `<cmath>` library. There is also a `log` function for natural logarithm (base e), and a `log10` function for base 10 logarithm.

```
double logValue = log2(8); // logValue will be 3  
double naturalLog = log(2.71828); // naturalLog will be approximately 1  
double logBase10 = log10(100); // logBase10 will be 2
```

-
- The max, min values in integer are `INT_MAX`, `INT_MIN` from the `<climits>` library.
 - For floating point numbers, use `FLT_MAX`, `FLT_MIN` from the `<cfloat>` library.
 - For long integers, use `LONG_MAX`, `LONG_MIN` from the `<climits>` library.
 - For double precision floating point numbers, use `DBL_MAX`, `DBL_MIN` from the `<double.h>` library.

Some built-in functions in C++, and their time complexities:

Function	Complexity
<code>min(a, b)</code>	$O(1)$
<code>max(a, b)</code>	$O(1)$
<code>swap(a, b)</code>	$O(1)$
<code>binary_search(arr, arr + n, x)</code>	$O(\log(n))$
<code>lower_bound(arr, arr + n, x)</code>	$O(\log(n))$
<code>upper_bound(arr, arr + n, x)</code>	$O(\log(n))$
<code>pow(a, n)</code>	$O(\log(n))$
<code>reverse(arr, arr + n)</code>	$O(n)$

Function	Complexity
<code>min_element(arr, arr + n)</code>	$O(n)$
<code>max_element(arr, arr + n)</code>	$O(n)$
<code>fill(arr, arr + n, x)</code>	$O(n)$
<code>count(arr, arr + n, x)</code>	$O(n)$
<code>find(arr, arr + n, x)</code>	$O(n)$
 <code>sort(arr, arr + n)</code>	 $O(n \log(n))$
<code>next_permutation(arr, arr + n)</code>	$O(n \cdot n!)$

- What is amortized analysis?